

Metodos numericos para localizar minimos de smoothness en la perdida de forecast

Nota tecnica del proyecto `paper_numerical-smoothness-selection`

Resumen

Esta nota desarrolla la parte numerica que queda escondida detras de encontrar los minimos locales de la perdida en smoothness. El procedimiento implementado no es una simple busqueda sobre una grilla. La tendencia penalizada y el forecast tienen primeras y segundas derivadas analiticas con respecto a λ ; estas derivadas se transforman a la coordenada normalizada S , se usa una busqueda adaptativa para detectar intervalos que pueden contener puntos estacionarios y, finalmente, cada raiz de la derivada se refina con Brent. Tambien se tratan explicitamente los endpoints $S = 0$ y $S = 1$, se deduplican soluciones numericamente equivalentes y se clasifican los puntos por el signo de la segunda derivada.

El objetivo es documentar exactamente la matematica y las decisiones numericas de las rutinas activas del repositorio.

Índice

1. Problema numerico

Para una configuracion fija

$$(d, L, h),$$

consideramos una ventana de entrenamiento

$$\mathbf{x} = (x_1, \dots, x_L)^\top$$

y un bloque futuro observado

$$\mathbf{z} = (z_1, \dots, z_h)^\top.$$

La tendencia penalizada es

$$\hat{\boldsymbol{\tau}}_\lambda = \arg \min_{\boldsymbol{\tau} \in \mathbb{R}^L} \{ \|\mathbf{x} - \boldsymbol{\tau}\|_2^2 + \lambda \|D_d \boldsymbol{\tau}\|_2^2 \}. \quad (1)$$

Sea

$$Q_d = D_d^\top D_d.$$

Entonces

$$\hat{\boldsymbol{\tau}}_\lambda = H_\lambda \mathbf{x}, \quad H_\lambda = (I + \lambda Q_d)^{-1}. \quad (2)$$

Despues de continuar la tendencia h pasos, obtenemos

$$\hat{\mathbf{z}}_\lambda.$$

La funcion objetivo de una Validation 1 es

$$F(\lambda) = \frac{1}{h} \|\mathbf{z} - \hat{\mathbf{z}}_\lambda\|_2^2. \quad (3)$$

La funcion que realmente exploramos es

$$\tilde{F}(S) = F(\lambda(S)), \quad 0 \leq S \leq 1.$$

La tarea numerica es recuperar todos los minimos locales relevantes

$$\mathcal{M} = \left\{ S : \tilde{F}_S(S) = 0, \quad \tilde{F}_{SS}(S) > 0 \right\},$$

mas los posibles minimos de frontera.

2. Representacion espectral del suavizador

Sea

$$Q_d = U \text{diag}(\delta_1, \dots, \delta_L) U^\top, \quad \delta_j \geq 0.$$

Para una penalizacion por diferencias de orden d , exactamente d eigenvalores son cero.

Definimos

$$\mathbf{c} = U^\top \mathbf{x}$$

y

$$\alpha_j(\lambda) = \frac{1}{1 + \lambda \delta_j}. \quad (4)$$

Entonces

$$\hat{\boldsymbol{\tau}}_\lambda = U \text{diag}(\alpha_j(\lambda)) \mathbf{c}. \quad (5)$$

La dependencia en λ queda separada en los escalares α_j . Esto permite obtener derivadas exactas sin diferencias finitas.

2.1. Primera y segunda derivada

De (??),

$$\alpha'_j(\lambda) = -\frac{\delta_j}{(1 + \lambda \delta_j)^2} = -\delta_j \alpha_j^2,$$

y

$$\alpha''_j(\lambda) = \frac{2\delta_j^2}{(1 + \lambda \delta_j)^3} = 2\delta_j^2 \alpha_j^3.$$

Por tanto,

$$\hat{\boldsymbol{\tau}}'_\lambda = U \text{diag}(-\delta_j \alpha_j^2) \mathbf{c}, \quad (6)$$

$$\hat{\boldsymbol{\tau}}''_\lambda = U \text{diag}(2\delta_j^2 \alpha_j^3) \mathbf{c}. \quad (7)$$

Equivalentemente,

$$\hat{\boldsymbol{\tau}}'_\lambda = -H_\lambda Q_d \hat{\boldsymbol{\tau}}_\lambda, \quad (8)$$

$$\hat{\boldsymbol{\tau}}''_\lambda = 2H_\lambda Q_d H_\lambda Q_d \hat{\boldsymbol{\tau}}_\lambda. \quad (9)$$

Estas son las identidades implementadas en `pure_trend_derivatives`.

3. Derivadas del forecast

La continuacion de diferencias finitas puede escribirse como un operador lineal

$$C_{d,h,L} \in \mathbb{R}^{h \times L}$$

tal que

$$\hat{\mathbf{z}}_\lambda = C_{d,h,L} \hat{\mathbf{r}}_\lambda. \quad (10)$$

Como $C_{d,h,L}$ no depende de λ ,

$$\hat{\mathbf{z}}'_\lambda = C_{d,h,L} \hat{\mathbf{r}}'_\lambda, \quad (11)$$

$$\hat{\mathbf{z}}''_\lambda = C_{d,h,L} \hat{\mathbf{r}}''_\lambda. \quad (12)$$

4. Derivadas exactas de la MSE

Sea

$$\mathbf{r}_\lambda = \mathbf{z} - \hat{\mathbf{z}}_\lambda.$$

La perdida es

$$F(\lambda) = \frac{1}{h} \mathbf{r}_\lambda^\top \mathbf{r}_\lambda.$$

Como

$$\mathbf{r}'_\lambda = -\hat{\mathbf{z}}'_\lambda,$$

se obtiene

$$F'(\lambda) = -\frac{2}{h} \mathbf{r}_\lambda^\top \hat{\mathbf{z}}'_\lambda, \quad (13)$$

y

$$F''(\lambda) = \frac{2}{h} \left[(\hat{\mathbf{z}}'_\lambda)^\top \hat{\mathbf{z}}'_\lambda - \mathbf{r}_\lambda^\top \hat{\mathbf{z}}''_\lambda \right]. \quad (14)$$

Asi, cada evaluacion produce

$$\boxed{F(\lambda), \quad F'(\lambda), \quad F''(\lambda)}.$$

No se aproximan estas derivadas mediante diferencias finitas. Esto evita escoger un paso numerico auxiliar y reduce el numero de forecasts necesarios.

5. Objetivo rolling y vectorizacion

Con varios rolling origins, sea $\mathcal{R}$ su conjunto y $\mathbf{r}_{r,\lambda}$ el residual del origin r . La implementacion usa

$$F_{\text{roll}}(\lambda) = \frac{\sum_{r \in \mathcal{R}} \|\mathbf{r}_{r,\lambda}\|_2^2}{N_{\text{score}}}, \quad N_{\text{score}} = \sum_{r \in \mathcal{R}} h_r. \quad (15)$$

Las derivadas se agregan de la misma forma. Para ventanas y horizontes fijos, el codigo prepara una sola vez el espectro de Q_d , las ventanas de training en la base espectral, el operador de forecast y todos los targets futuros.

Si B contiene las ventanas historicas en coordenadas espectrales y

$$G = C_{d,h,L} U,$$

entonces las predicciones de todos los origins pueden escribirse simultaneamente como

$$\widehat{Z}_\lambda = \left(B \odot \alpha(\lambda)^\top \right) G^\top. \quad (16)$$

Para las derivadas se reemplaza α por α' o α'' .

6. La coordenada normalizada de smoothness

Para $d > 0$,

$$S(\lambda) = 1 - \frac{1}{L-d} \sum_{j:\delta_j > 0} \frac{1}{1 + \lambda\delta_j}. \quad (17)$$

Sus primeras dos derivadas son

$$S'(\lambda) = \frac{1}{L-d} \sum_{j:\delta_j > 0} \frac{\delta_j}{(1 + \lambda\delta_j)^2} > 0, \quad (18)$$

y

$$S''(\lambda) = -\frac{2}{L-d} \sum_{j:\delta_j > 0} \frac{\delta_j^2}{(1 + \lambda\delta_j)^3} < 0. \quad (19)$$

Por tanto,

$$S : [0, \infty] \rightarrow [0, 1]$$

es monotona creciente con

$$S(0) = 0, \quad S(\infty) = 1.$$

7. Regla de la cadena de λ a S

Definimos

$$\widetilde{F}(S) = F(\lambda(S)).$$

Como $S'(\lambda) > 0$,

$$\frac{d\lambda}{dS} = \frac{1}{S'(\lambda)}.$$

Entonces

$$\widetilde{F}_S(S) = \frac{F'(\lambda)}{S'(\lambda)}, \quad (20)$$

y

$$\widetilde{F}_{SS}(S) = \frac{F''(\lambda)}{[S'(\lambda)]^2} - \frac{F'(\lambda)S''(\lambda)}{[S'(\lambda)]^3}. \quad (21)$$

Como $S'(\lambda) > 0$,

$$\widetilde{F}_S(S) = 0 \iff F'(\lambda) = 0. \quad (22)$$

Ademas, en un punto estacionario,

$$F'(\lambda) = 0,$$

y por tanto

$$\widetilde{F}_{SS}(S) = \frac{F''(\lambda)}{[S'(\lambda)]^2}. \quad (23)$$

El signo de la curvatura se conserva. Un minimo en λ sigue siendo un minimo en S .

8. Como se calcula $\lambda(S)$

Para $0 < S < 1$, la implementacion invierte (??) por biseccion monotona. Se resuelve

$$\phi(\lambda) = S(\lambda) - S_0 = 0. \quad (24)$$

El algoritmo actual es:

1. Si $S_0 = 0$, regresar $\lambda = 0$.
2. Si $S_0 = 1$, regresar $\lambda = \infty$.
3. Iniciar

$$\lambda_{lo} = 0, \quad \lambda_{hi} = 1.$$

4. Mientras

$$S(\lambda_{hi}) < S_0,$$

usar

$$\lambda_{hi} \leftarrow 10\lambda_{hi},$$

hasta encerrar el target o alcanzar 10^{16} .

5. Bisecar el intervalo y conservar la mitad que contiene el target.
6. Detener cuando

$$|S(\lambda_{mid}) - S_0| < 10^{-11}$$

o despues de 100 iteraciones.

La biseccion explota una propiedad fuerte del problema: la monotonicidad de $S(\lambda)$ esta demostrada, por lo que la raiz interior es unica.

9. Por que no basta con un optimizador local

Si $\tilde{F}(S)$ fuera unimodal, un metodo local podria ser suficiente. Pero aqui queremos recuperar

$$S_1^*, S_2^*, \dots, S_j^*,$$

no solamente

$$\arg \min_S \tilde{F}(S).$$

Un unico Newton, BFGS o golden-section search puede encontrar un minimo y omitir las demas cuencas. Por ello la implementacion separa

descubrimiento de regiones con raices $\longrightarrow$ refinamiento de cada raiz.

10. Dominio interior de busqueda

La busqueda derivativa se hace sobre

$$0 \leq S \leq 1 - \eta, \quad \eta = 10^{-6}. \quad (25)$$

El punto $S = 1$ corresponde a $\lambda = \infty$ y se trata por separado como endpoint.

11. Malla inicial y refinamiento de endpoints

La rutina inicia con

$$m_0 = 9$$

puntos uniformes:

$$S_i = \frac{i}{m_0 - 1}(1 - \eta), \quad i = 0, \dots, m_0 - 1.$$

No es una grilla final. Solo es el esqueleto de una busqueda adaptativa.

La compactificacion puede comprimir estructura cerca de $S = 1$, y en menor grado cerca de $S = 0$. Por eso se insertan seis niveles diadicos dentro de las dos celdas de frontera. Si w es el ancho de una celda inicial, se agregan distancias

$$\frac{w}{2}, \frac{w}{2^2}, \dots, \frac{w}{2^6}$$

desde cada extremo.

12. Refinamiento adaptativo

Definimos

$$g(S) = \tilde{F}_S(S), \quad q(S) = \tilde{F}_{SS}(S).$$

Para un intervalo $[a, b]$, sea

$$m = \frac{a + b}{2}.$$

Evaluuamos

$$g_a = g(a), \quad g_m = g(m), \quad g_b = g(b),$$

y

$$q_a = q(a), \quad q_m = q(m), \quad q_b = q(b).$$

El intervalo se subdivide si alguno de cuatro diagnosticos indica estructura no resuelta.

12.1. La derivada gira

Se refina si

$$(g_m - g_a)(g_b - g_m) < 0. \tag{26}$$

12.2. La curvatura cambia de signo

Con

$$\tau_q = 10^{-8},$$

se refina si

$$\min(q_a, q_m, q_b) < -\tau_q \quad \text{y} \quad \max(q_a, q_m, q_b) > \tau_q. \tag{27}$$

12.3. La derivada central esta cerca de cero

Sea

$$\tau_g = 10^{-8}, \quad \rho = 0.2,$$

y

$$G_{\text{edge}} = \text{máx}(|g_a|, |g_b|, \tau_g).$$

Se refina si

$$|g_m| \leq \text{máx}(\tau_g, \rho G_{\text{edge}}). \quad (28)$$

12.4. Cambio de signo oculto

Tambien se refina si los extremos tienen el mismo signo pero una de las dos mitades cambia de signo:

$$\text{sign}(g_a) = \text{sign}(g_b),$$

junto con

$$g_a g_m < 0 \quad \text{o} \quad g_m g_b < 0.$$

12.5. Parada

El refinamiento se detiene al alcanzar

$$\text{max_depth} = 8$$

o cuando

$$b - a \leq 10^{-3}.$$

13. Construcción de brackets

Después del refinamiento, los puntos evaluados se ordenan:

$$S_1 < S_2 < \dots < S_M.$$

Un punto muestreado se considera raíz candidata si

$$|g(S_i)| \leq 10^{-8}. \quad (29)$$

Además, un par consecutivo define un bracket si

$$g(S_i)g(S_{i+1}) < 0. \quad (30)$$

Cada bracket se resuelve con Brent.

14. Metodo de Brent

La implementación usa

$$\text{scipy.optimize.brentq}$$

para resolver

$$g(S) = \tilde{F}_S(S) = 0.$$

Brent conserva un bracket y combina tres mecanismos.

14.1. Biseccion

$$S_{\text{bis}} = \frac{a+b}{2}.$$

Es el paso robusto que garantiza reducir el bracket.

14.2. Secante

Con dos puntos, la propuesta es

$$S_{\text{sec}} = b - g(b) \frac{b-a}{g(b)-g(a)}. \quad (31)$$

14.3. Interpolacion cuadratica inversa

Con tres puntos adecuados, Brent aproxima S como funcion cuadratica de g y evalua esa aproximacion en $g = 0$. Este paso puede ser mucho mas rapido que biseccion cerca de una raiz regular.

14.4. Salvaguarda

Si una propuesta de secante o interpolacion no satisface las condiciones de seguridad del metodo, se vuelve a biseccion. Asi se combina

$$\boxed{\text{robustez de bracket} + \text{velocidad de interpolacion}}.$$

La tolerancia actual es

$$\text{root_xtol} = 10^{-10}.$$

15. Por que no Newton como buscador principal

Newton sobre $g(S) = 0$ seria

$$S_{k+1} = S_k - \frac{g(S_k)}{q(S_k)}. \quad (32)$$

Puede ser muy rapido cerca de una raiz, pero:

1. necesita una semilla por cada raiz;
2. puede saltar de una cuenca a otra;
3. es problematico cuando $q(S_k)$ es pequeno;
4. no descubre por si solo cuantos puntos estacionarios existen.

El objetivo de este proyecto es recuperar multiples minimos. Por eso se usa

refinamiento adaptativo $\rightarrow$ brackets $\rightarrow$ Brent.

16. Deduplicacion y clasificacion

Una misma raiz puede detectarse como punto casi estacionario y tambien mediante un bracket. Despues de ordenar las raices se fusionan aquellas con separacion menor que

$$\tau_{\text{merge}} = \text{máx}(10 \text{ extttroot_xtol}, 10^{-8}). \quad (33)$$

Con los parametros actuales,

$$\tau_{\text{merge}} = 10^{-8}.$$

Para cada raiz deduplicada S^* , se evalua

$$q(S^*) = \tilde{F}_{SS}(S^*).$$

Con $\tau_q = 10^{-8}$,

$$\begin{cases} q(S^*) > \tau_q & \Rightarrow \text{minimum}, \\ q(S^*) < -\tau_q & \Rightarrow \text{maximum}, \\ |q(S^*)| \leq \tau_q & \Rightarrow \text{flat}. \end{cases} \quad (34)$$

El tracking conserva los puntos **minimum** y los minimos de frontera.

17. Minimos de frontera

Los endpoints no necesitan satisfacer $F_S = 0$. El experimento evalua

$$\tilde{F}(0), \quad \tilde{F}(1),$$

y usa

$$\epsilon_b = 0.002$$

como sonda interior.

Se acepta $S = 0$ si

$$\tilde{F}(0) \leq \tilde{F}(\epsilon_b), \quad (35)$$

y $S = 1$ si

$$\tilde{F}(1) \leq \tilde{F}(1 - \epsilon_b). \quad (36)$$

Estas son condiciones unilaterales de minimo.

18. Separacion de candidatos

Despues de reunir minimos interiores y de frontera, el experimento de tracking elimina candidatos numericamente demasiado cercanos.

Se ordenan por menor Validation-1 loss. Un candidato S_b se descarta si ya se retuvo S_a con

$$|S_b - S_a| < \epsilon_{\text{cand}}, \quad \epsilon_{\text{cand}} = 0.02. \quad (37)$$

Esta operacion ocurre dentro de una sola superficie. No promedia smoothness ni fusiona ramas temporales.

En el primer origin se inicializan como maximo cinco de esos minimos.

19. Estabilidad numerica del espectro

En aritmetica exacta,

$$\text{nullity}(D_d^\top D_d) = d.$$

En punto flotante, los ceros estructurales pueden aparecer como numeros muy pequenos positivos o negativos.

El codigo canonicaliza el espectro:

$$\delta_1 = \dots = \delta_d = 0,$$

y para el resto

$$\delta_j \leftarrow \max(\delta_j, 0).$$

Esto evita que un error de eigensolver en un valor que deberia ser cero sea amplificado cuando λ es grande.

20. Tratamiento de $\lambda = \infty$

Para $S = 1$, no se calcula literalmente $1 + \infty \cdot 0)^{-1}$ *se usa ellimite* $\alpha_j(\infty) = \begin{cases} 1, & \delta_j = 0, \\ 0, & \delta_j > 0. \end{cases}$ Las derivadas se fijan en cero. Asi

$$\hat{\tau}_\infty = P_{\ker D_d} \mathbf{x}.$$

21. Cache de evaluaciones

Durante la subdivision, un punto medio se convierte despues en extremo de dos intervalos. Para evitar recalculo, se guarda

$$S \mapsto (\lambda, F, F_\lambda, F_{\lambda\lambda}, F_S, F_{SS}).$$

Cada valor de S se evalua una sola vez. El contador `n_evaluations_` mide precisamente cuantos smoothness distintos fueron evaluados.

22. Complejidad

La eigendescomposicion densa inicial cuesta, en orden,

$$O(L^3).$$

Pero esa operacion se prepara una sola vez por configuracion.

Despues, cada nuevo λ reutiliza el espectro y el operador de forecast. La evaluacion trabaja con pesos espectrales y multiplicaciones matriciales vectorizadas sobre todos los rolling origins.

Por tanto, evaluar M candidatos no equivale a resolver desde cero M problemas penalizados completos.

23. Por que seguimos usando una grilla densa como diagnostico

Ningun muestreo finito puede certificar, sin hipotesis adicionales, que encontro todas las raices de una funcion suave arbitraria. Una oscilacion suficientemente estrecha puede quedar entre puntos muestreados.

Por eso los benchmarks comparan

busqueda adaptativa derivativa vs. perfil denso de referencia.

La grilla densa sirve para revisar:

- cuantos minimos se recuperaron;
- si se omitio una cuenca estrecha;
- el error de localizacion de cada S^* ;
- el numero de evaluaciones ahorradas.

La grilla es una referencia numerica, no el algoritmo principal.

24. Algoritmo completo

Para una superficie fija:

1. Preparar el objetivo rolling y su algebra espectral.
2. Crear nueve puntos iniciales en $[0, 1 - 10^{-6}]$.
3. Agregar seis niveles diadicos cerca de ambos endpoints.
4. Para cada nuevo S :
 - a) invertir $S \mapsto \lambda$ por biseccion;
 - b) evaluar $F, F_\lambda, F_{\lambda\lambda}$;
 - c) transformar a (F, F_S, F_{SS}) .
5. Refinar los intervalos cuya derivada o curvatura sugiera estructura.
6. Guardar puntos con $|F_S| \leq 10^{-8}$.
7. Crear brackets donde F_S cambia de signo.
8. Resolver cada bracket con Brent hasta $extttxtol = 10^{-10}$.
9. Deduplicar raices a escala 10^{-8} .
10. Clasificar con F_{SS} .
11. Revisar $S = 0$ y $S = 1$ como minimos unilaterales.
12. Separar candidatos dentro de 0.02.
13. Inicializar hasta cinco ramas para el tracking temporal.

25. Flujo matematico

El calculo de los minimos puede resumirse como

$$\begin{array}{c}
 \boxed{S \xrightarrow{\text{biseccion}} \lambda(S) \xrightarrow{\text{espectro}} \hat{\tau}, \hat{\tau}', \hat{\tau}''} \\
 \\
 \boxed{\xrightarrow{C_{d,h,L}} \hat{z}, \hat{z}', \hat{z}'' \xrightarrow{\text{MSE}} F, F_\lambda, F_{\lambda\lambda}} \\
 \\
 \boxed{\xrightarrow{\text{regla de la cadena}} F_S, F_{SS} \xrightarrow{\text{refinamiento}} \text{brackets} \xrightarrow{\text{Brent}} S_j^*}.
 \end{array}$$

Finalmente,

$$S_j^* \in \mathcal{M} \iff F_S(S_j^*) = 0 \quad \text{y} \quad F_{SS}(S_j^*) > 0,$$

salvo los minimos unilaterales de frontera.

26. Relacion con el tracking temporal

Todo lo anterior ocurre dentro de una sola ventana. El tracking temporal documentado en `tracked_minimum_smoothness_selection.tex` toma el conjunto

$$\mathcal{M}_{d,r} = \{S_{d,1,r}, \dots, S_{d,J_r,r}\}$$

y lo asocia con los minimos de la ventana siguiente mediante

$$|S_{d,j,r+1} - S_{d,j,r}| \leq \varepsilon_{\text{track}}.$$

Por tanto existen dos problemas distintos:

A: localizar todos los minimos de una superficie

y

B: asociar esos minimos entre ventanas

.

Esta nota documenta A. La nota de tracking desarrolla B y la seleccion final mediante Validation 2.

27. Conclusion

La seleccion numerica de smoothness no es una exploracion ciega de S . La implementacion explota la estructura lineal del suavizador para obtener derivadas analiticas, transforma esas derivadas exactamente a la coordenada normalizada, adapta el muestreo a la geometria observada y resuelve cada raiz detectada con un metodo con bracket.

La idea central es

usar derivadas para descubrir donde buscar + usar Brent para localizar cada raiz robustamente

.

Esto hace posible recuperar multiples minimos locales con muchas menos evaluaciones que una grilla muy densa, mientras que los perfiles densos permanecen como diagnostico independiente de cobertura.